Avaliação da qualidade de dormentes de concreto baseado em visão computacional e redes neurais convolucionais

Pedro Montes Melo * **Michel Carlo Rodrigues Leles** **

* *Universidade Federal de São João Del Rei (UFSJ), MG (e-mail: pedrommelo01@gmail.com).*
** *Departamento de Tecnologia em Engenharia Civil, Computação, Automação, Telemática e Humanidades (DTECH), MG (e-mail: mleles@ufsj.edu.br).*

Abstract: This work presents an automated system model to classify the quality of concrete railway sleepers into two categories (GOOD and POOR), utilizing machine learning approaches based on classic algorithms with HOG-based feature extraction and convolutional neural networks (CNNs) operating with transfer learning. Three classic models (Random Forest, SVM, and XGBoost) and five deep learning architectures (MobileNet V2, ResNet50, Inception V3, DenseNet121, and EfficientNetB0) were evaluated. Among the classic approaches, SVM stood out with an accuracy of 97.78%. The CNN-based models demonstrated similar performance, with MobileNetV2 reaching the highest accuracy (98.89%). The feasibility analysis for embedded systems revealed that, despite their greater parameter complexity, the CNNs maintained a robust processing rate on CPU. The results indicate the technical potential of computer vision to support automated sleeper inspection.

Resumo: Este trabalho apresenta um modelo de sistema automatizado para classificar a qualidade de dormentes ferroviários de concreto em duas categorias (GOOD e POOR), utilizando abordagens de aprendizado de máquina baseadas em algoritmos clássicos com extração de características via HOG e redes neurais convolucionais (CNNs) operando com transfer learning. Foram avaliados três modelos clássicos (Random Forest, SVM e XGBoost) e cinco arquiteturas de deep learning (MobileNet V2, ResNet50, Inception V3, DenseNet121 e EfficientNetB0). Entre as abordagens clássicas, o SVM destacou-se com acurácia de 97,78%. Os modelos baseados em CNN demonstraram desempenho similar, com a arquitetura MobileNetV2 alcançando o melhor desempenho (acurácia de 98,89%). A análise de viabilidade para sistemas embarcados revelou que, apesar de sua maior complexidade de parâmetros, as CNNs mantiveram uma taxa de processamento robusta em CPU. Os resultados indicam o potencial técnico da visão computacional para apoiar a automatização da inspeção visual de dormentes.

Keywords: Railway Inspection; Concrete Sleepers; Deep Learning; Convolutional Neural Networks (CNNs); Embedded Systems

Palavras-chaves: Inspeção Ferroviária; Dormentes de Concreto; Aprendizado Profundo; Redes Neurais Convolucionais; Sistemas Embarcados

1. INTRODUÇÃO

O setor ferroviário desempenha papel estratégico na logística de transportes de cargas em escala global, sendo responsável pela movimentação de commodities agrícolas, minerais e produtos industrializados com eficiência energética superior aos modais rodoviário e aéreo. O modal ferroviário representa 21,5% da matriz de transporte de cargas no Brasil, movimentando aproximadamente 540 milhões de toneladas úteis anualmente em uma malha de 31.000 km de extensão (Associação Nacional dos Transportadores Ferroviários, 2024).

A manutenção preventiva e preditiva da infraestrutura ferroviária constitui fator crítico para garantia de segurança operacional, disponibilidade de via e redução de custos com paradas não programadas. Entre os componentes da superestrutura ferroviária, os dormentes, elementos estruturais que sustentam os trilhos e distribuem cargas dinâmicas para o lastro, apresentam deterioração progressiva decorrente de solicitações mecânicas cíclicas, variações térmicas, infiltração de água e ataque químico ao concreto. A norma ISO 22480-1:2022 estabelece requisitos técnicos para projeto, materiais e controle de qualidade de dormentes de concreto, especificando ensaios mecânicos para assegurar capacidade de resistir a carregamentos repetitivos e durabilidade adequada ao longo da vida útil projetada, tipicamente 50 anos em condições normais de operação (International Organization for Standardization, 2022).

Avanços recentes em visão computacional e aprendizado profundo têm viabilizado desenvolvimento de sistemas automatizados de inspeção visual com desempenho comparável ou superior à avaliação humana em domínios específicos. Redes neurais convolucionais (CNNs), arquiteturas de aprendizado profundo projetadas para processamento de dados visuais estruturados em grades bidimensionais, demonstraram capacidade de extrair representações hierárquicas de características visuais diretamente de imagens brutas, dispensando engenharia manual de features e alcançando acurácias superiores a 95% em tarefas de classificação de imagens (He et al., 2016; Huang et al., 2017). A técnica de transfer learning, que consiste em reutilizar redes neurais pré-treinadas em datasets de larga escala como ImageNet (14 milhões de imagens, 1000 classes) e adaptá-las a domínios específicos mediante fine-tuning, tem-se mostrado particularmente eficaz em cenários com disponibilidade limitada de dados rotulados, reduzindo drasticamente tempo de treinamento e requisitos computacionais (Kornblith et al., 2019).

Aplicações de inteligência artificial à inspeção de infraestrutura ferroviária têm sido objeto de investigação crescente na literatura científica e em implementações operacionais. Em 2024, Kaewunruen et al. desenvolveram sistema baseado em aprendizado profundo para prognóstico e diagnóstico de características complexas de trincas em dormentes de concreto, alcançando acurácia de 98% na classificação de padrões de fissuração (Kaewunruen et al., 2024). Tang e Quian conduziram revisão sistemática em inspeção ferroviária inteligente, identificando CNNs como abordagem predominante para detecção automatizada de defeitos em componentes da via permanente (Tang and Qian, 2024). Em termos de implementações práticas, a Deutsche Bahn, operadora ferroviária alemã, implantou sistema de inspeção automatizada baseado em visão computacional capaz de processar imagens de dormentes a velocidades de até 140 km/h, inspecionando mais de 62 milhões de dormentes de concreto instalados em sua malha (Deutsche Luft- und Raumfahrtzentrum, 2025). Nos Estados Unidos, sistemas automatizados realizam aproximadamente 3,5 milhões de inspeções diárias de componentes ferroviários, demonstrando viabilidade técnica e econômica da automação em larga escala (Association of American Railroads, 2025).

Diante desse contexto, este trabalho busca responder às seguintes questões:

(1) **RQ1:** Modelos de aprendizado profundo baseados em redes neurais convolucionais são capazes de classificar automaticamente a qualidade de dormentes de concreto a partir de imagens?
(2) **RQ2:** Como o desempenho das arquiteturas convolucionais se compara ao de modelos clássicos de aprendizado de máquina baseados em descritores manuais?
(3) **RQ3:** Quais modelos apresentam melhor equilíbrio entre desempenho preditivo e viabilidade computacional para uma possível aplicação embarcada?

2. REVISÃO BIBLIOGRÁFICA

A revisão de literatura deste trabalho foi organizada em duas seções: uma com foco em inspeções inteligentes no contexto ferroviário, com uso de arquiteturas de deep learning, e outra focando em estudos e implementações de arquiteturas de deep learning de modo geral, não necessariamente relacionadas com essa linha de pesquisa.

2.1 Arquiteturas de deep learning aplicadas à inspeção ferroviária

A aplicação de técnicas de aprendizado profundo à inspeção automatizada de infraestrutura ferroviária tem sido objeto de investigação crescente nos últimos anos, motivada pela necessidade de substituir métodos manuais por sistemas escaláveis, consistentes e eficientes. Arquiteturas da família YOLO (*You Only Look Once*) têm demonstrado desempenho superior em tarefas de detecção de componentes ferroviários em tempo real, sendo amplamente adotadas devido à combinação de acurácia e velocidade de inferência.

Wang et al. (2020) implementaram YOLOv2 para detecção em tempo real de trilhos, parafusos e grampos em vias férreas, alcançando mAP (*mean Average Precision*) de 93%. Os autores conduziram análise comparativa com YOLO original (92% mAP) e YOLOv3 (89% mAP), concluindo que YOLOv3 apresenta arquitetura excessivamente complexa para este domínio específico, resultando em overfitting durante treinamento. Experimentos demonstraram que YOLOv2 oferece melhor balanceamento entre precisão e velocidade para aplicações de inspeção ferroviária móvel (Wang et al., 2020).

Passos (2021) investigou a aplicação de redes neurais convolucionais para identificação automática de defeitos superficiais em trilhos ferroviários a partir de imagens. O estudo utilizou um conjunto de 670 imagens contendo trilhos sem defeitos e com defeitos do tipo Head Checking, Flaking e Squat, coletadas em trechos operacionais da ferrovia da MRS Logística. As imagens foram utilizadas no treinamento de diferentes arquiteturas de redes neurais, com aplicação de técnicas de pré-processamento e aumento de dados para melhorar o desempenho dos modelos. Os resultados indicaram que arquiteturas pré-treinadas utilizando transfer learning, especialmente a InceptionV3, apresentaram melhor desempenho na classificação automática dos defeitos analisados (Passos, 2021).

Ferdousil et al. (2024) desenvolveram sistema combinando VGG19, MobileNetV3 e ResNet50 com mecanismo de adaptação dinâmica de pesos para detecção de defeitos em trilhos e rodas ferroviárias, alcançando 99% de acurácia em validação e teste, com precisão de 0,99 e recall e F1-score de 0,98. Abordagem demonstrou consistência superior a modelos individuais, mitigando problema de seleção de arquitetura única para domínios com características visuais heterogêneas. Trabalho evidencia que combinação de arquiteturas com diferentes capacidades representacionais VGG19 para extração de features de alto nível, MobileNetV3 para eficiência e ResNet50 para captura de padrões complexos supera desempenho de detectores isolados (Ferdousil et al., 2024).

2.2 Arquiteturas de deep learning para classificação e detecção de defeitos

Arafin et al. (2024) criaram dataset diversificado contendo 4.087 imagens de trincas e 1.100 de spalling em estru-

turas de concreto, avaliando três arquiteturas, VGG19, ResNet50 e InceptionV3 como classificadores CNN e dois modelos encoder-decoder (U-Net e PSPNet) com quatro backbones distintos (VGG19, ResNet50, InceptionV3, EfficientNetB3) para segmentação semântica. InceptionV3 alcançou melhor desempenho em classificação com 91,98% de acurácia, enquanto U-Net com backbones EfficientNetB3 e InceptionV3 apresentou segmentação superior para trincas e spalling, respectivamente. Estudo demonstra que arquiteturas com convoluções multiescala (Inception) apresentam vantagem em captura de defeitos com características visuais em múltiplas resoluções (Arafin et al., 2024).

Feng et al. (2019) aplicaram transfer learning com CNNs para detecção de danos estruturais, evidenciando que redes pré-treinadas em ImageNet transferem eficientemente características visuais de baixo nível (bordas, texturas, gradientes) para domínios específicos de inspeção civil, mesmo com datasets reduzidos. Experimentos comparando treinamento do zero versus fine-tuning de redes pré-treinadas demonstraram redução de 60% no tempo de convergência e melhoria de 12% em acurácia mediante transfer learning, validando estratégia adotada neste trabalho (Feng et al., 2019).

Abubakr et al. (2024) introduziram abordagem baseada em aprendizado profundo para classificar e detectar cinco defeitos comuns em pontes de concreto armado mediante modelos Xception e Vanilla baseados em CNNs. Estudo comparativo demonstrou eficácia da classificação para imagens de inspeção estrutural, com modelo Xception devido ao número de parâmetros mantendo capacidade representacional. Trabalho evidencia viabilidade de classificação simultânea de múltiplos tipos de defeitos, abordagem relevante para inspeção de componentes estruturais (Abubakr et al., 2024).

3. METODOLOGIA

A metodologia utilizada nesse trabalho está estruturada de acordo com o diagrama apresentado na Figura 1.



Figura 1. Fluxograma de metodologia empregada.

3.1 Critérios de Classificação de Qualidade

Para a rotulação das imagens, recomenda-se seguir normas técnicas nacionais e internacionais, literatura científica especializada e critérios operacionais específicos de companhias ferroviárias. As duas classes (GOOD e POOR) representam estados de deterioração estrutural, com critérios simplificados para facilitar a rotulação manual visual.

Os critérios adotados baseiam-se em normas como:

(1) ABNT NBR 11709:2015, que especifica requisitos para projeto e fabricação de dormentes de concreto (Associação Brasileira de Normas Técnicas, 2015);
(2) ABNT NBR 6118:2014, que estabelece tolerâncias para fissuras em estruturas de concreto armado (Associação Brasileira de Normas Técnicas, 2014);
(3) ISO 22480-1:2022, que define critérios técnicos e procedimentos de controle para materiais constituintes e dormentes acabados (International Organization for Standardization, 2022).

A classe GOOD contém dormentes estruturalmente íntegros, aptos para operação contínua sem necessidade de intervenção. Devem conter as seguintes características: ausência de fraturas ou trincas, regiões de apoio do trilho íntegras, armadura não exposta e deformação < 2 mm.

A classe POOR contém dormentes com comprometimento estrutural significativo que exigem substituição imediata ou em curto prazo. para evitar riscos. Os dormentes desta classe podem possuir trincas longitudinais com w $> 1{,}5$ mm, penetração completa, trincas transversais propagadas através de $> 2/3$ da altura ou fraturas completas, armadura exposta com cobrimento < 5 mm ou corrosão ativa visível, deformação > 5 mm e perda de seção estrutural comprometendo capacidade portante.

A Tabela 1 sintetiza os critérios estabelecidos.

Tabela 1. Critérios para rotulação dos dormentes

Parâmetro	GOOD	POOR
Trincas longitudinais (mm)	Ausente	$> 1{,}5$
Trincas transversais (altura)	Ausente	$> 2/3$
Armadura exposta (mm)	Ausente	< 5 / corrosão
Deformação (mm)	< 2	> 5

O conjunto de imagens utilizado neste trabalho foi obtido com a rotulação previamente realizada.

3.2 Aquisição do conjunto de imagens

O conjunto de dados utilizado consiste em 600 imagens digitais de dormentes ferroviários de concreto capturadas sob condições reais de operação. As imagens foram obtidas a partir do conjunto de dados público Utami (2023) disponibilizado na plataforma Kaggle. O conjunto de imagens original foi adaptado, foram mantidas apenas duas categorias e foram reorganizados dormentes entre as categorias e incluindo novas imagens autorais. O dataset contém registros fotográficos de dormentes de concreto em diferentes condições de integridade estrutural.

As condições de captura incluem diversidade de iluminação, diferentes ângulos e variados estados de conservação

superficial, assegurando representatividade das condições operacionais encontradas em campo.

Cada imagem foi examinada individualmente, identificando-se os defeitos visíveis para confirmar a rotulação.

A distribuição final do dataset foi: GOOD: 305 imagens (50,8%) e POOR: 295 imagens (49,2%), totalizando 600 imagens.



Figura 2. Classe GOOD



Figura 3. Classe POOR

3.3 Pré-processamento das imagens

O pré-processamento foi composto por sete operações sequenciais.

3.3.1. Redução de ruído por filtro bilateral Inicialmente, foi aplicada uma etapa de redução de ruído por meio do filtro bilateral. Diferentemente de filtros convencionais de suavização, que podem borrar regiões de transição, o filtro bilateral considera simultaneamente a proximidade espacial entre os pixels e a similaridade de suas intensidades. Dessa forma, a suavização ocorre prioritariamente em regiões visualmente homogêneas, enquanto descontinuidades relevantes, como bordas, fissuras, fraturas e limites de lascamentos, tendem a ser preservadas.Ressalta-se que o filtro teve como finalidade reduzir pequenas variações e ruídos de aquisição, sem substituir ou eliminar características estruturais visíveis. A imagem resultante dessa etapa foi utilizada como entrada para o procedimento subsequente de remoção automática das bordas.

3.3.2. Remoção automática de bordas uniformes Após a redução de ruído, foi aplicado um procedimento de recorte automático para remover regiões periféricas uniformes que não contribuíam diretamente para a classificação. Essas regiões poderiam corresponder, por exemplo, a bordas pretas, brancas ou de coloração aproximadamente constante decorrentes da aquisição, exportação ou organização das imagens.

O método utilizado não procurou diretamente pelo objeto dormente nem realizou segmentação semântica. Em vez disso, avaliou a uniformidade das faixas externas da imagem. Para essa finalidade, a imagem foi inicialmente convertida do espaço de cores BGR para escala de cinza. Em seguida, linhas e colunas foram analisadas progressivamente a partir das extremidades superior, inferior, esquerda e direita. A busca foi limitada a, no máximo, 50 pixels ou 25% da respectiva dimensão da imagem.

Depois da identificação dos limites de recorte, foi mantida uma margem adicional de cinco pixels ao redor da região considerada relevante. Essa margem de segurança foi utilizada para diminuir a possibilidade de remoção de informações localizadas próximas às extremidades detectadas. A finalidade dessa etapa foi diminuir a presença de regiões periféricas sem informação e aumentar a proporção da área útil da imagem antes do ajuste de contraste e do redimensionamento.

3.3.3. Ajuste de contraste por CLAHE Na terceira etapa, foi empregado o método CLAHE, do inglês Contrast Limited Adaptive Histogram Equalization, com o objetivo de reduzir os efeitos de variações locais de iluminação e melhorar a distinção visual de detalhes presentes na superfície dos dormentes. Essa estratégia é adequada para imagens obtidas em campo, nas quais um mesmo dormente pode apresentar simultaneamente áreas iluminadas, regiões sombreadas e superfícies com diferentes condições de conservação. Os ajustes foram aplicados exclusivamente ao canal L, que representa a luminosidade da imagem. O emprego do CLAHE teve como finalidade favorecer a visualização de variações locais de textura e intensidade associadas a fissuras, lascamentos, fraturas e exposição de armaduras. Entretanto, o procedimento não cria novas informações estruturais nem garante, isoladamente, a melhor identificação desses defeitos. Sua função no pipeline foi padronizar parcialmente as diferenças de contraste existentes nas fotografias e fornecer entradas visualmente mais homogêneas aos modelos avaliados.

3.3.4. Redimensionamento das imagens Todas as imagens foram redimensionadas para a resolução de 224 × 224 pixels. O redimensionamento preservou a proporção original das imagens e utilizou preenchimento com pixels pretos quando necessário, evitando deformações.

3.3.5. Normalização dos pixels Após o redimensionamento, as imagens foram mantidas em representação na faixa [0,255]. Para as CNNs, a transformação numérica final foi realizada por meio da função correspondente a cada arquitetura pré-treinada. MobileNetV2 e InceptionV3 aplicaram transformação para a faixa aproximada [-1,1]; ResNet50 e DenseNet121 empregaram as transformações específicas estabelecidas pelas respectivas implementações e EfficientNetB0 utilizou o pré-processamento definido pela implementação da arquitetura. Para os modelos clássicos, a extração HOG foi realizada separadamente, seguida de padronização das características e redução dimensional por PCA.

3.3.6. Multiplicação dos dados Para mitigar overfitting (problema comum quando a capacidade representacional da rede excede a diversidade do conjunto de treinamento) foram aplicadas técnicas de aumento de dados (*data augmentation*) exclusivamente durante o treinamento, gerando variações das imagens originais sem alterar suas classes. As transformações aplicadas foram:

(1) Três rotações ortogonais (90°, 180° e 270°);
(2) Ajustes de contraste (incremento de 30% e decremento de 20%).
(3) Espelhamentos (horizontal, vertical, além de combinações de espelhamento com rotação de 90°);
(4) Ajustes de brilho (incremento e decremento de 20



Figura 4. Demonstração do pré processamento

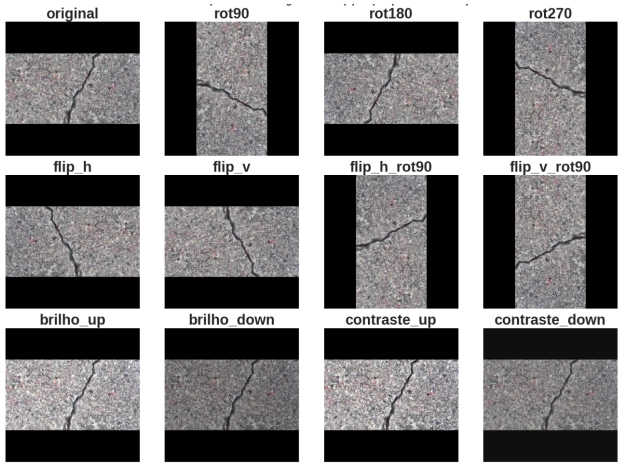


Figura 5. Demonstração da multiplicação de dados

Essas transformações foram implementadas de forma offline. As imagens modificadas foram geradas uma única vez e salvas fisicamente em uma pasta. Conjunto de validação e teste não receberam multiplicação de imagens, sendo avaliados apenas com imagens originais (redimensionadas e normalizadas) para não enviesar o desempenho. Após a multiplicação, o conjunto totalizou 5231 imagens.

3.3.7. Organização dos conjuntos de treinamento, validação e teste O conjunto de dados utilizado neste trabalho foi previamente organizado em três subconjuntos independentes: treinamento, validação e teste. Essa organização foi definida na própria estrutura de diretórios da base de imagens, de modo que não foi realizada uma nova divisão durante a execução do pipeline. O subconjunto de treinamento foi empregado no ajuste dos parâmetros dos modelos. O subconjunto de validação foi utilizado para acompanhar o desempenho durante o treinamento, selecionar a melhor fase de treinamento e auxiliar na definição das configurações dos modelos. Por sua vez, o subconjunto de teste foi reservado para a avaliação final do desempenho em imagens não utilizadas no ajuste dos modelos.

3.4 Seleção das arquiteturas clássicas e de CNNs

3.4.1. Seleção de arquiteturas clássicas Três arquiteturas clássicas de aprendizado de máquina foram selecionadas para avaliação comparativa, representando diferentes paradigmas de classificação e amplamente consolidados na literatura.

3.4.1.1. Random Forest O Random Forest é um algoritmo composto por múltiplas árvores de decisão treinadas de forma independente sobre subconjuntos aleatórios dos dados. A decisão final é obtida por votação entre as árvores, aumentando a robustez e reduzindo problemas de sobreajuste. A arquitetura foi selecionada por sua elevada capacidade de generalização, baixo custo de ajuste e ampla utilização em tarefas de classificação supervisionada. Além disso, o modelo apresenta boa interpretabilidade e tolerância a ruídos, características relevantes em aplicações industriais e problemas envolvendo variabilidade visual nas imagens. Sua inclusão no estudo permite avaliar a capacidade de modelos baseados em árvores de decisão em distinguir padrões estruturais presentes nos dormentes (Breiman, 2001).

3.4.1.2. Support Vector Machine (SVM) O Support Vector Machine (SVM) é um algoritmo fundamentado na maximização da margem de separação entre classes em espaços de alta dimensionalidade. O modelo consegue representar fronteiras de decisão não lineares, tornando-se eficiente em problemas de classificação de imagens. Historicamente, foi amplamente empregado em aplicações de visão computacional antes da popularização das arquiteturas de Deep Learning. O SVM foi incluído neste trabalho devido à sua elevada capacidade de generalização, eficiência em conjuntos de dados moderados e forte relevância histórica em tarefas de reconhecimento de padrões visuais. (Cortes and Vapnik, 1995).

3.4.1.3. XGBoost O XGBoost é um algoritmo baseado em árvores de decisão, desenvolvido com foco em eficiência computacional, paralelização e regularização avançada. O modelo constrói sucessivas árvores de decisão de forma iterativa, corrigindo progressivamente os erros cometidos pelas árvores anteriores. Essa estratégia proporciona elevada capacidade preditiva e excelente desempenho em problemas supervisionados. O XGBoost foi selecionado por representar uma das abordagens clássicas mais modernas e competitivas da literatura, sendo amplamente utilizado em competições de aprendizado de máquina e aplicações industriais. Sua inclusão permite investigar se métodos clássicos baseados em boosting conseguem competir com arquiteturas profundas na tarefa de classificação da qualidade estrutural dos dormentes (Chen and Guestrin, 2016).

A Tabela 2 sintetiza as características dos modelos clássicos de aprendizado de máquina avaliados.

Tabela 2. Modelos clássicos avaliados

Modelo	**Abordagem**
Random Forest	Agrupamento de árvores de decisão
SVM	Busca da melhor linha de separação
XGBoost	Construção sequencial de modelos para corrigir erros

3.4.2. Seleção de arquiteturas de CNNs Cinco arquiteturas de redes neurais convolucionais foram selecionadas para avaliação comparativa, representando diferentes

abordagens de projeto e diferentes relações entre desempenho e custo computacional. Todas as redes utilizam pesos pré-treinados no conjunto de dados ImageNet (aproximadamente 14 milhões de imagens distribuídas em 1000 classes), permitindo aplicar transfer learning. Dessa forma, características visuais básicas, como bordas, texturas e padrões, previamente aprendidas em um domínio amplo, podem ser reutilizadas no problema específico de inspeção de dormentes.

3.4.2.1. MobileNetV2 Arquitetura projetada para uso em dispositivos móveis e sistemas embarcados, utilizando técnicas que reduzem o número de operações computacionais sem comprometer significativamente o desempenho. Possui aproximadamente 3,4 milhões de parâmetros treináveis, o que reduz o uso de memória e o tempo de processamento em comparação com arquiteturas mais complexas. Essa rede foi selecionada por sua viabilidade de implementação em plataformas embarcadas de baixo custo, como Raspberry Pi, possibilitando aplicações de inspeção em campo sem dependência constante de servidores remotos. Além disso, sua inclusão na análise permite avaliar se arquiteturas mais leves mantêm bom desempenho na tarefa de classificação da qualidade estrutural dos dormentes (Sandler et al., 2018).

3.4.2.2. ResNet50 Rede neural profunda com 50 camadas que utiliza conexões residuais, também chamadas de skip connections, permitindo que a informação atravesse diferentes camadas da rede sem se degradar. Essa estratégia facilita o treinamento de redes muito profundas e melhora a capacidade de aprendizado. A arquitetura possui aproximadamente 25,6 milhões de parâmetros e foi incluída no estudo por ser amplamente utilizada em aplicações de visão computacional. Sua avaliação permite investigar se uma rede mais profunda consegue identificar melhor padrões complexos de defeitos estruturais em dormentes ou se a alta complexidade pode levar a problemas de sobreajuste devido ao tamanho limitado do conjunto de dados (He et al., 2016).

3.4.2.3. InceptionV3 Arquitetura baseada em módulos Inception, que analisam a imagem em diferentes escalas ao mesmo tempo por meio de convoluções paralelas com filtros de diferentes tamanhos. Esse mecanismo permite capturar padrões pequenos e grandes simultaneamente. A rede possui aproximadamente 23,8 milhões de parâmetros e utiliza técnicas que reduzem o custo computacional sem comprometer o desempenho. Essa arquitetura foi incluída no estudo por sua capacidade de identificar características em múltiplas escalas, o que é relevante para a detecção de defeitos em dormentes. Esses defeitos podem variar desde microfissuras superficiais até fraturas maiores que ocupam grande parte da imagem. Dessa forma, é possível avaliar se essa abordagem multiescala oferece vantagens em relação a arquiteturas que analisam as imagens em uma única escala (Szegedy et al., 2016).

3.4.2.4. DenseNet121 Arquitetura em que cada camada recebe como entrada as saídas de todas as camadas anteriores, criando uma rede densamente conectada. Essa estratégia permite reutilizar características já aprendidas e melhora o fluxo de informação durante o treinamento. A versão DenseNet121 possui 121 camadas e cerca de 8 milhões de parâmetros. Essa arquitetura foi selecionada por sua eficiência no aprendizado com conjuntos de dados menores, já que o reaproveitamento de características pode melhorar o treinamento e reduzir redundâncias no modelo. Assim, sua inclusão permite avaliar se esse tipo de conectividade melhora a capacidade de identificar defeitos estruturais em dormentes (Huang et al., 2017).

3.4.2.5. EfficientNetB0 Arquitetura desenvolvida por meio de técnicas de busca automática de arquitetura de redes neurais (Neural Architecture Search – NAS). O modelo equilibra três aspectos principais da rede (profundidade, largura e resolução da imagem) de forma conjunta para melhorar a eficiência do aprendizado. A versão EfficientNetB0 possui aproximadamente 5,3 milhões de parâmetros e apresenta boa relação entre desempenho e custo computacional. Essa arquitetura foi incluída no estudo por representar uma abordagem moderna de projeto de redes neurais, permitindo avaliar se modelos otimizados automaticamente podem apresentar melhor desempenho do que arquiteturas tradicionais projetadas manualmente (Tan and Le, 2019).

A Tabela 3 sintetiza as características das cinco arquiteturas.

Tabela 3. Arquiteturas de CNN avaliadas

Arquitetura	Parâmetros
MobileNetV2	3,4M
ResNet50	25,6M
InceptionV3	23,8M
DenseNet121	8,0M
EfficientNetB0	5,3M

Para todas as arquiteturas foi utilizada a estratégia de transfer learning, aproveitando modelos previamente treinados no conjunto de dados ImageNet. A partir dessa base, foram adicionadas novas camadas responsáveis por processar as informações da imagem e realizar a classificação. Também foi aplicada uma técnica de regularização (dropout de 0,5) para reduzir o risco de o modelo aprender padrões muito específicos do conjunto de dados. A camada final possui duas saídas, correspondendo às duas classes de qualidade dos dormentes (GOOD e POOR).

3.5 Procedimento de treinamento

Os experimentos foram executados no ambiente Google Colab, utilizando recursos computacionais disponibilizados na nuvem.

3.5.1. Arquiteturas clássicas Três modelos clássicos foram implementados para fins comparativos Random Forest, Support Vector Machine (SVM) e XGBoost. Diferentemente de CNNs que aprendem features, modelos clássicos utilizam descritores visuais extraídos manualmente.

O fluxo segue duas etapas sequenciais:

3.5.1.1. Extração de features Foi utilizado Histogram of Oriented Gradients (HOG), técnica que detecta bordas e formas das imagens, divide a imagem em pequenas regiões e conta em qual direção as mudanças de cor (bordas) ocorrem em cada região. O resultado é um vetor numérico descrevendo as principais bordas e formas presentes na imagem (Dalal and Triggs, 2005).

3.5.1.2. Normalização e Redução As features HOG foram normalizadas e reduzidas via Principal Component Analysis (PCA), técnica que mantém apenas as características mais importantes e descarta detalhes ruidosos. Essa redução acelera o treinamento dos modelos (Jolliffe, 2002).

Após isso, é feita a configuração dos hiperparâmetros do modelo para iniciar o treinamento.

3.5.2. Arquiteturas CNN O treinamento das cinco arquiteturas seguiu procedimento padronizado implementado em duas fases sequenciais, conforme estratégia de fine-tuning amplamente adotada na literatura de transfer learning:

3.5.2.1. Fase 1: Treinamento das Camadas Superiores (Épocas 1-30) Nesta fase inicial, todas as camadas convolucionais da rede base (pré-treinada no ImageNet) foram congeladas, impedindo atualização de seus pesos. Apenas as camadas densas adicionadas ao topo da rede foram treinadas. Isso foi adotado pois as camadas iniciais de CNNs aprendem detectores genéricos (bordas, gradientes, texturas) que são transferíveis para diferentes conjunto de imagens. Além disso, treinar todas as camadas simultaneamente com dataset limitado induz overfitting. Dessa forma, o treinamento restrito às camadas superiores permite adaptação rápida da rede à tarefa específica deste trabalho (classificação de dormentes) enquanto preserva conhecimento transferido das camadas convolucionais. Para essa fase, foi adotada uma taxa de aprendizado de 1×10^{-4}.

3.5.2.2. Fase 2: Ajuste fino das camadas finais da rede (épocas 31–50) Após a estabilização do treinamento na fase inicial, algumas das últimas camadas da rede base foram liberadas para continuar o aprendizado. Essas camadas são responsáveis por identificar padrões mais complexos nas imagens. O número de camadas liberadas variou conforme a arquitetura, correspondendo 10% das camadas mais próximas da saída da rede.

Durante essa etapa, a taxa de aprendizado foi reduzida para 1×10^{-5}, permitindo que os pesos da rede fossem ajustados de forma mais gradual. Essa redução evita alterações muito bruscas nos parâmetros já aprendidos durante o pré-treinamento.

O objetivo dessa etapa é permitir que o modelo se adapte melhor às características específicas dos dormentes ferroviários, melhorando a capacidade da rede em identificar defeitos estruturais nas imagens.

Este procedimento dividido em duas fases balanceia estabilidade (Fase 1: camadas superiores convergem sem perturbar base pré-treinada) e adaptabilidade (Fase 2: rede completa refina-se ao projeto específico).

Ao término das duas fases, a melhor época de cada fase foi avaliada no conjunto de validação. O modelo final foi selecionado prioritariamente pela maior acurácia de validação e, em caso de empate, pela menor perda de validação. Dessa forma, a versão final poderia corresponder à Fase 1 ou à Fase 2, evitando que uma eventual degradação durante o fine-tuning comprometesse o desempenho final.

3.6 Hiperparâmetros e estratégias de regularização

3.6.1. Arquiteturas clássicas

3.6.1.1. Random Forest Como estratégia de regularização, os hiperparâmetros do modelo Random Forest foram definidos com base na literatura e posteriormente ajustados por meio de uma busca em grade (grid search) utilizando o conjunto de validação, uma vez que o conjunto de dados já havia sido previamente dividido, mantendo o conjunto de teste totalmente isolado durante todo o processo. Para escolha da melhor combinação, foi utilizado o F1-Score ponderado como critério de seleção. A configuração inicial foi baseada nos estudos de Kumar et al. (2022), que utilizaram 100 a 200 árvores em problemas de classificação de defeitos estruturais, e de Géron (2022), que justifica o uso de cada hiperparâmetro e fornece estratégias para redução de overfitting.

Foram empregadas estratégias de regularização por meio dos hiperparâmetros *min_samples_split*, *min_samples_leaf* e *max_depth*, que restringem, respectivamente, a divisão de nós contendo poucas amostras, a criação de folhas excessivamente específicas e a profundidade máxima das árvores. Também foi utilizada a seleção aleatória de atributos em cada divisão (*max_features*), que melhora a sua capacidade de generalização. Para esse parâmetro, foi selecionado o valor 0.5, correspondente à avaliação de aproximadamente 50% dos atributos disponíveis em cada divisão. A normalização e regularização via PCA foi mantida em 85%.

A Tabela 4 apresenta a configuração final dos hiperparâmetros selecionados para o modelo.

Tabela 4. Configuração de Hiperparâmetros – Random Forest

Hiperparâmetro	Valor
`n_estimators`	100
`max_depth`	12
`min_samples_split`	10
`min_samples_leaf`	8
`max_features`	0.7
`max_samples`	0.7
`max_leaf_nodes`	50
`ccp_alpha`	0.001

3.6.1.2. SVM A estratégia de regularização do SVM consistiu na seleção dos hiperparâmetros por meio de um grid search utilizando exclusivamente o conjunto de validação, mantendo o conjunto de teste isolado durante todo o processo de ajuste. Foram avaliadas diferentes combinações dos parâmetros C e γ , que ajustam o quanto o modelo penaliza erros no conjunto de treinamento e o alcance de influência de cada amostra, respectivamente, além do uso de *class_weight='balanced'*, com o objetivo

de reduzir possíveis efeitos do desbalanceamento entre as classes. Essa relação entre C e γ é destacada por Hsu et al. (2003), que recomendam a busca conjunta desses hiperparâmetros em classificadores com kernel RBF. A escolha da melhor configuração foi baseada no maior valor de F1-Score ponderado. A normalização e regularização via PCA foi mantida em 90%. A configuração final selecionada está representada na Tabela 5.

Os valores estão representados na Tabela 5.

Tabela 5. Configuração de Hiperparâmetros – Support Vector Machine (SVM)

Hiperparâmetro	Valor
`kernel`	rbf
`C`	5
`gamma`	scale

3.6.1.3. XGBoost Para reduzir o risco de sobreajuste, foram combinadas estratégias de controle estrutural, penalização dos pesos e subamostragem. A profundidade máxima das árvores foi avaliada entre 4 e 5, enquanto o parâmetro *gamma* assumiu os valores 0, 1 e 2. O *min_child_weight* foi fixado em 2, restringindo a formação de folhas sustentadas por pouca informação. Também foram aplicadas penalizações L1 e L2, com *reg_alpha*=0,5 e *reg_lambda*=1. A subamostragem foi controlada pelos parâmetros *subsample* e *colsample_bytree*, avaliados entre 0,8 e 0,9, enquanto *colsample_bylevel* e *colsample_bynode* foram mantidos em 0,8. Esses mecanismos buscaram reduzir a dependência do modelo em relação a observações e características específicas do conjunto de treinamento. Foram comparados os *boosters gbtree* e *dart*. Para o *dart*, foram avaliadas taxas de descarte de árvores de 0 e 0,05, com *skip_drop*=0,5. O treinamento utilizou taxa de aprendizado de 0,01, limite de 500 estimadores e parada antecipada após 30 iterações sem melhoria da *mlogloss* de validação. Devido ao custo computacional da avaliação exaustiva, a otimização foi realizada por busca aleatória com o método *ParameterSampler*. Foram amostradas 20 configurações. A configuração final foi selecionada pelo F1-score ponderado de validação, penalizado pela diferença entre os resultados de treinamento e validação:

$$\text{Score} = F1_{val} - 0{,}3 \cdot (F1_{treino} - F1_{val}) \tag{1}$$

Esse critério favoreceu configurações com melhor desempenho de validação e menor diferença em relação ao treinamento. A capacidade de generalização foi posteriormente verificada no conjunto de teste. A normalização e regularização via PCA foi mantida em 95%.

A configuração final está representada na Tabela 6.

Tabela 6. Configuração de Hiperparâmetros – XGBoost

Hiperparâmetro	**Valor**
`booster`	`dart`
`n_estimators`	500
`max_depth`	4
`learning_rate`	0.01
`min_child_weight`	2
`gamma`	2
`subsample`	0.8
`colsample_bytree`	0.8
`colsample_bylevel`	0.8
`colsample_bynode`	0.8
`reg_alpha`	0.5
`reg_lambda`	1
`rate_drop`	0
`skip_drop`	0.5
`grow_policy`	`depthwise`
`max_delta_step`	0
`base_score`	0.33
`early_stopping_rounds`	30

3.6.2. Arquiteturas CNN

3.6.2.1. Função de perda Categorical Cross-Entropy Definida como:

$$\mathcal{L} = -\frac{1}{N}\sum_{i=1}^{N}\sum_{c=1}^{C} y_{i,c}\log(\hat{y}_{i,c}) \tag{2}$$

onde N representa o número de amostras utilizadas em cada etapa do treinamento, chamada de *batch*. Um *batch* corresponde a um pequeno conjunto de imagens processadas pela rede de cada vez durante o treinamento, em vez de utilizar todo o conjunto de dados simultaneamente.

O termo $C = 2$ representa o número de classes do problema (GOOD e POOR). O valor $y_{i,c}$ indica a classe correta da amostra, representada no formato *one-hot*. Nesse formato, cada classe é representada por um vetor em que apenas uma posição possui valor 1 (indicando a classe correta) e todas as demais posições possuem valor 0. Por exemplo, para duas classes possíveis, um dormente classificado como GOOD pode ser representado por $(1, 0)$ e POOR seria $(0, 1)$.

Já $\hat{y}_{i,c}$ representa a probabilidade estimada pelo modelo para cada classe. Essas probabilidades são obtidas por meio da função *softmax*, que transforma os valores de saída da rede neural em probabilidades entre 0 e 1, de forma que a soma das probabilidades de todas as classes seja igual a 1. Assim, a classe com maior probabilidade é interpretada como a previsão do modelo.

A função Categorical Cross-Entropy mede o quanto as probabilidades previstas pelo modelo diferem da classe correta. Durante o treinamento, o objetivo do modelo é reduzir esse valor, ajustando seus parâmetros para produzir previsões cada vez mais próximas das classes verdadeiras. A escolha pela função se baseou em literaturas correlatas (Goodfellow et al., 2016; Chen et al., 2022).

3.6.2.2. Otimizador Adam (Adaptive Moment Estimation)
O otimizador é o algoritmo responsável por atualizar os parâmetros da rede neural durante o treinamento, ajustando-os gradualmente para reduzir o erro do modelo. Neste trabalho foi utilizado o otimizador Adam com taxa de aprendizado inicial $\alpha = 1 \times 10^{-4}$ na Fase 1 e $\alpha = 1 \times 10^{-5}$ na Fase 2. A taxa de aprendizado controla o tamanho dos ajustes feitos nos parâmetros da rede a cada etapa do treinamento.

Os parâmetros $\beta_1 = 0.9$ e $\beta_2 = 0.999$ controlam como o algoritmo considera o histórico das atualizações anteriores para tornar o processo de aprendizado mais estável. Já o termo $\epsilon = 1 \times 10^{-7}$ é um pequeno valor utilizado para evitar problemas numéricos durante os cálculos.

O otimizador Adam foi escolhido por ser amplamente utilizado em modelos de aprendizado profundo, apresentando bom desempenho e estabilidade no treinamento de redes neurais convolucionais(Kumar et al., 2022; Chen et al., 2022).

3.6.2.3. Tamanho do batch: 32 amostras O valor de 32 foi escolhido por representar um bom equilíbrio entre estabilidade no treinamento e eficiência computacional. Batches maiores tendem a produzir atualizações mais estáveis, enquanto batches menores aumentam o número de atualizações realizadas durante o treinamento. O valor de 32 é amplamente utilizado na literatura de aprendizado profundo e transfer learning para conjuntos de dados de tamanho moderado(Goodfellow et al., 2016; Chen et al., 2022).

3.6.2.4. Número máximo de épocas: 50 Uma época corresponde a uma passagem completa do modelo por todas as imagens do conjunto de treinamento. O limite de 50 épocas foi definido como valor suficiente para permitir que as arquiteturas aprendam os padrões presentes nos dados e com base em literaturas correlatas (Kumar et al., 2022; Simonyan and Zisserman, 2014).

Entretanto, foi utilizado o método *Early Stopping*, que interrompe o treinamento automaticamente caso o desempenho no conjunto de validação pare de melhorar. Dessa forma, evita-se treinamento desnecessário e reduz-se o risco de sobreajuste do modelo aos dados de treinamento.

3.6.2.5. Estratégias de Regularização Regularização refere-se a um conjunto de técnicas utilizadas durante o treinamento de modelos de aprendizado de máquina para evitar o *overfitting*, isto é, quando o modelo aprende excessivamente os detalhes do conjunto de treinamento e perde a capacidade de generalizar para novos dados. Neste trabalho foram utilizadas quatro técnicas de regularização.

Dropout Foi aplicada uma taxa de 0,5 na camada densa intermediária adicionada após a rede base pré-treinada (descrita na etapa de transfer learning). Essa camada é responsável por combinar as características extraídas pela rede convolucional antes da etapa final de classificação.

Durante o treinamento, aproximadamente metade dos neurônios dessa camada é desativada de forma aleatória a cada iteração. Essa estratégia foi aplicada tanto na Fase 1 quanto na Fase 2 do treinamento. O objetivo é evitar que o modelo passe a depender excessivamente de poucos neurônios específicos, forçando a rede a aprender diferentes combinações de características. Com isso, reduz-se o risco de o modelo se ajustar demais aos dados de treinamento (*overfitting*), melhorando sua capacidade de generalização para novas imagens.

Dropout foi implementado por mostrar efetividade em trabalhos correlatos(Simonyan and Zisserman, 2014; Chen et al., 2022).

Regularização L2 (Weight Decay) Foi aplicado um fator $\lambda = 0.01$ aos pesos das camadas densas adicionadas na etapa de transfer learning (descritas anteriormente na arquitetura do modelo). Essa técnica adiciona um termo de penalização à função de perda:

$$\mathcal{L}_{total} = \mathcal{L}_{CE} + \lambda \sum_{l} \|W_l\|_2^2 \tag{3}$$

Essa penalização reduz a possibilidade de os pesos assumirem valores muito elevados durante o treinamento. Como resultado, o modelo tende a aprender padrões mais gerais nas imagens, em vez de memorizar características específicas do conjunto de treinamento. Esta técnica é amplamente aplicada em trabalhos que utilizam o transfer learning(Kumar et al., 2022; Chen et al., 2022).

Early Stopping O Early Stopping monitorou a acurácia de validação, com paciência de dez épocas. Como a restauração automática de pesos foi desativada, foi utilizado um *callback* personalizado para salvar o melhor modelo. O checkpoint priorizou a maior acurácia de validação e, em caso de empate dentro da tolerância estabelecida, selecionou a menor perda de validação.. Esta técnica foi utilizada por ser configuração padrão em transfer learning (Simonyan and Zisserman, 2014; Chen et al., 2022).

ReduceLROnPlateau Também foi utilizado um mecanismo de ajuste automático da taxa de aprendizado durante o treinamento. Quando a perda no conjunto de validação permanecia sem melhoria por 5 épocas consecutivas, a taxa de aprendizado era reduzida por um fator de 0,5, com valor mínimo de 1×10^{-7}. Essa redução permite que o treinamento continue realizando ajustes menores nos parâmetros da rede, ajudando o modelo a refinar o aprendizado quando o processo começa a se estabilizar. A escolha de permanência de 5 épocas foi baseada em literaturas correlatas (Goodfellow et al., 2016; Chen et al., 2022).

A Tabela 7 sintetiza configuração completa.

3.7 Métricas de avaliação

O desempenho foi avaliado mediante métricas padrão de classificação, calculadas sobre conjunto de teste isolado.

Tabela 7. Configuração de hiperparâmetros e regularização - CNNs

Hiperparâmetro	Valor
Função de perda	Categorical Cross-Entropy
Otimizador	Adam
Taxa aprendizado Fase 1	1×10^{-4}
Taxa aprendizado Fase 2	1×10^{-5}
β_1 (Adam)	0.9
β_2 (Adam)	0.999
Tamanho do batch	32
Épocas máximas	50
Dropout	0.5
L2 regularization	0.01
Early Stopping paciência	10 épocas
ReduceLR fator	0.5
ReduceLR paciência	5 épocas

3.7.1. Acurácia Global

$$Acc = \frac{TP + TN}{TP + TN + FP + FN} \tag{4}$$

onde TP (True Positives), TN (True Negatives), FP (False Positives) e FN (False Negatives) são calculados para cada classe e agregados. Representa proporção de predições corretas sobre total de amostras.

3.7.2. Precisão (Precision)

$$Precision = \frac{TP}{TP + FP} \tag{5}$$

Proporção de predições positivas que são corretas. Para classificação de dormentes, alta precisão indica baixa taxa de falsos alarmes (dormentes classificados erroneamente como defeituosos).

3.7.3. Recall (Sensibilidade)

$$Recall = \frac{TP}{TP + FN} \tag{6}$$

Proporção de positivos reais que foram corretamente identificados. Para dormentes, alto recall na classe POOR é crítico: falha em detectar dormente severamente danificado (falso negativo) implica riscos de segurança operacional.

3.7.4. F1-Score

$$F1 = 2 \cdot \frac{Precision \cdot Recall}{Precision + Recall} \tag{7}$$

Balanceia as métricas de precisão e recall. F1-Score ponderado foi calculado, atribuindo pesos proporcionais ao suporte de cada classe.

3.7.5. Curva de Treinamento Gráficos de acurácia e perda nos conjuntos de treinamento e validação ao longo das épocas. A comparação entre as curvas de treinamento e validação ajuda a identificar possíveis problemas de aprendizado. Quando o modelo apresenta bom desempenho no conjunto de treinamento, mas desempenho significativamente inferior no conjunto de validação, pode ocorrer *overfitting*, ou seja, o modelo passa a se ajustar excessivamente aos dados de treinamento e perde capacidade de generalização.

Por outro lado, quando tanto o desempenho no treinamento quanto na validação permanecem baixos ou não apresentam melhoria significativa ao longo das épocas, ocorre o chamado *underfitting*. Nesse caso, o modelo não consegue aprender adequadamente os padrões presentes nos dados, indicando que a arquitetura ou os parâmetros de treinamento podem não ser suficientes para representar o problema de forma adequada.

Todas as métricas foram calculadas mediante funções do scikit-learn, assegurando implementações validadas e amplamente aceitas pela comunidade científica.

3.8 Avaliação de viabilidade para sistemas embarcados

Para investigar a viabilidade de implementação em plataformas embarcadas de baixo custo, foram avaliadas as seguintes métricas de eficiência computacional:

(a) Número de parâmetros treináveis: quantidade total de pesos do modelo, indicando sua complexidade.
(b) Tamanho do modelo: tamanho do arquivo do modelo treinado em MB.
(c) Tempo de treinamento: tempo que o modelo leva para concluir o treinamento.
(d) Throughput (FPS): quantidade de imagens que o modelo consegue processar por segundo.

A análise comparativa dessas métricas permite equilibrar acurácia e eficiência computacional, auxiliando na escolha da arquitetura mais adequada para implementação em plataformas embarcadas.

4. RESULTADOS

Os códigos utilizados para treinamento, avaliação e comparação dos modelos estão disponíveis no repositório do projeto (Melo, 2026). Esta seção apresenta os resultados obtidos no treinamento e avaliação das arquiteturas para classificação de dormentes de concreto. A análise está organizada nas seguintes partes:

(a) desempenho geral das arquiteturas no conjunto de teste;
(b) evolução do treinamento e diagnóstico de overfitting;
(c) viabilidade de aplicação em sistemas embarcados.

Todos os resultados apresentados correspondem a métricas calculadas sobre o conjunto de teste (15% do conjunto total de dados), que permaneceu completamente separado durante todo o processo de treinamento e validação.

4.1 Resultados das arquiteturas clássicas

4.1.1. Desempenho geral das arquiteturas clássicas A Tabela 8 sintetiza o desempenho das três arquiteturas avaliadas segundo as métricas de classificação.

Tabela 8. Desempenho das arquiteturas clássicas no conjunto de teste

Arquitetura	**Acc**	**Prec**	**Rec**	**F1**
Random Forest	96,67%	96,69%	96,67%	96,67%
SVM	97,78%	97,78%	97,78%	97,78%
XGBoost	96,67%	96,69%	96,67%	96,67%

O SVM apresentou o melhor desempenho entre os modelos clássicos avaliados. Esse resultado pode estar relacionado à sua capacidade de encontrar uma boa separação entre as classes, mesmo quando as características das imagens

apresentam certa sobreposição. O uso do kernel RBF também contribuiu para o resultado, permitindo que o modelo identificasse relações mais complexas entre as características extraídas pelo HOG. Além disso, a configuração utilizada, com (C=5) e (*gamma=scale*), apresentou uma boa adaptação aos dados utilizados no estudo, resultando em uma classificação mais eficiente das diferentes condições dos dormentes.

4.1.2. Desempenho por classe individual A Tabela 9 detalha precision, recall e F1-Score por classe para a arquitetura de melhor desempenho (*SVM*).

Tabela 9. Desempenho por classe da arquitetura *SVM*

Classe	Precision	Recall	F1-Score	Imagens
GOOD	97,78%	97,78%	97,78%	45
POOR	97,78%	97,78%	97,78%	45

4.1.3. Curvas de treinamento e diagnóstico de overfitting As curvas de aprendizado foram geradas utilizando proporções crescentes do conjunto de treinamento (10%, 25%, 40%, 55%, 70%, 85% e 100%). Para cada proporção, o modelo foi treinado utilizando uma amostra correspondente do conjunto de treinamento. O desempenho foi avaliado tanto sobre a parcela utilizada no treinamento quanto sobre um conjunto de validação fixo, mantido inalterado em todos os pontos da curva. Essa abordagem permite analisar a evolução do desempenho do modelo conforme aumenta a quantidade de dados disponíveis, auxiliando na identificação de possíveis sinais de overfitting ou underfitting.

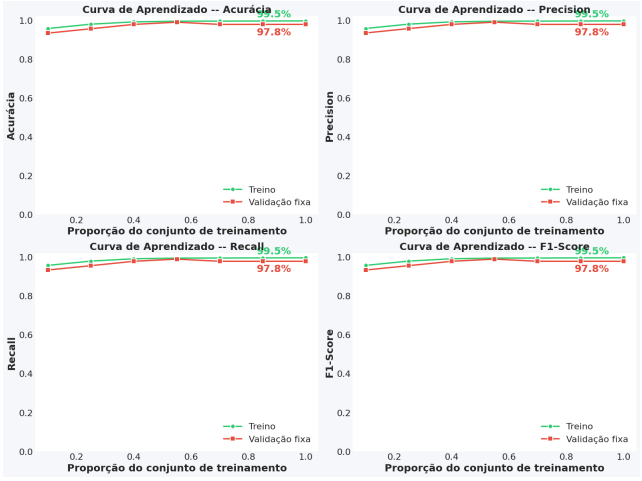


Figura 6. Curvas de treinamento e validação *Random Forest*

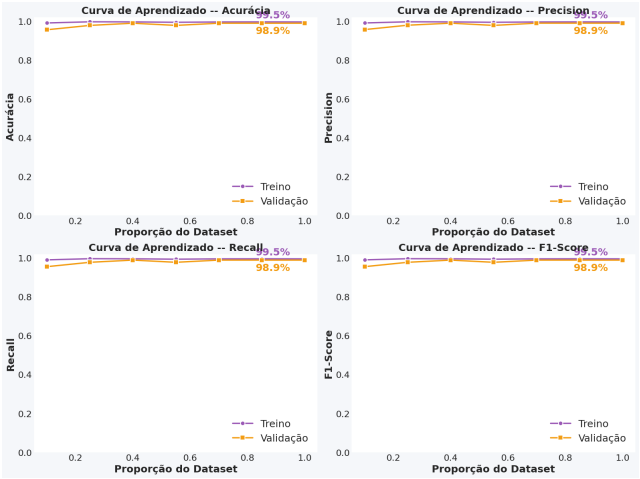


Figura 7. Curvas de treinamento e validação *XGBoost*

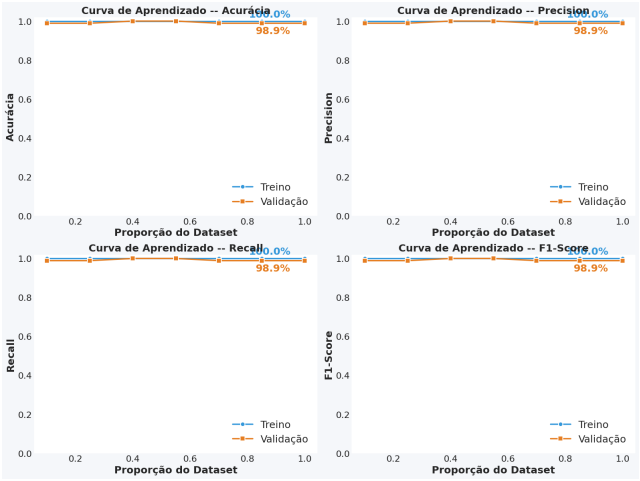


Figura 8. Curvas de treinamento e validação *SVM*

4.2 Resultados das arquiteturas CNN

4.2.1. Desempenho geral das arquiteturas A Tabela 10 sintetiza o desempenho das cinco arquiteturas avaliadas segundo seis as métricas de classificação.

Tabela 10. Desempenho das arquiteturas no conjunto de teste

Arquitetura	Acc	Prec	Rec	F1
MobileNetV2	98,89%	98,91%	98,89%	98,89%
ResNet50	96,67%	96,69%	96,67%	96,67%
InceptionV3	95,56%	95,56%	95,56%	95,56%
DenseNet121	96,67%	96,69%	96,67%	96,67%
EfficientNetB0	95,56%	95,56%	95,56%	95,56%

4.2.2. Desempenho por classe individual A Tabela 11 detalha precision, recall e F1-Score por classe para a arquitetura de melhor desempenho (*MobileNetV2*).

Tabela 11. Desempenho por classe da arquitetura *MobileNetV2*

Classe	Precision	Recall	F1-Score	Imagens
GOOD	100%	97,78%	98,88%	45
POOR	97,83%	100%	98,9%	45

4.2.3. Curvas de treinamento e diagnóstico de overfitting

As figuras abaixo apresentam a evolução das métricas de avaliação nos conjuntos de treinamento e validação ao longo das épocas para as cinco arquiteturas. Análise dessas curvas permite diagnosticar overfitting ou underfitting.

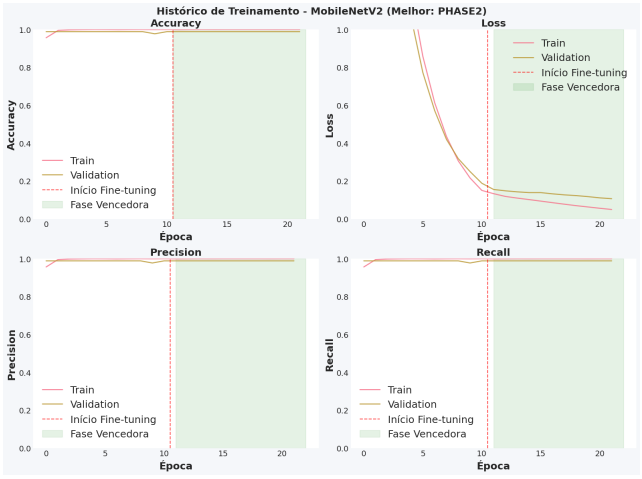


Figura 9. Curvas de treinamento e validação *MobileNetV2*

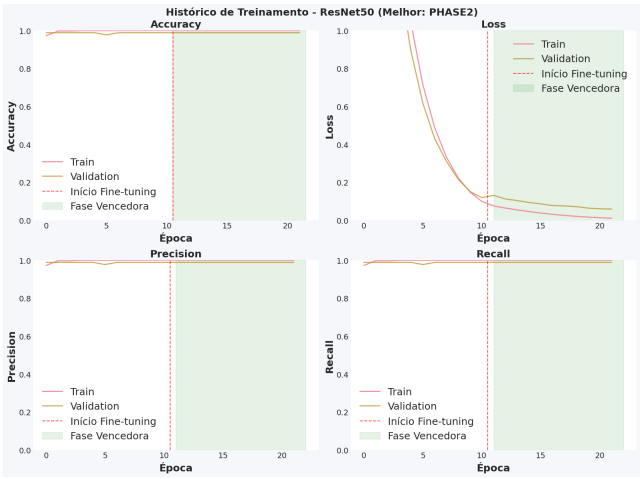


Figura 10. Curvas de treinamento e validação *ResNet50*

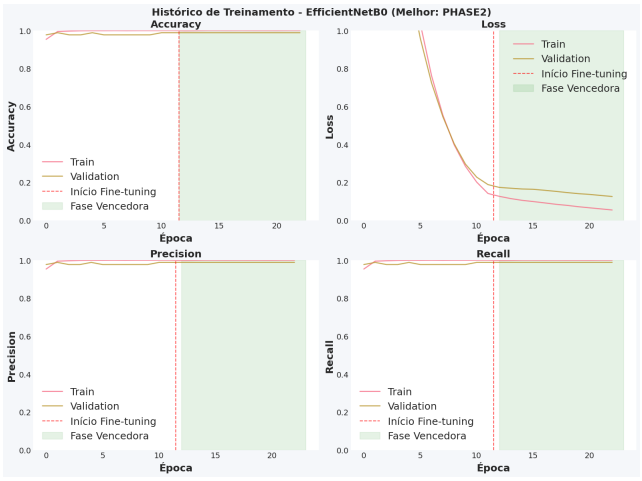


Figura 11. Curvas de treinamento e validação *EfficientNetB0*

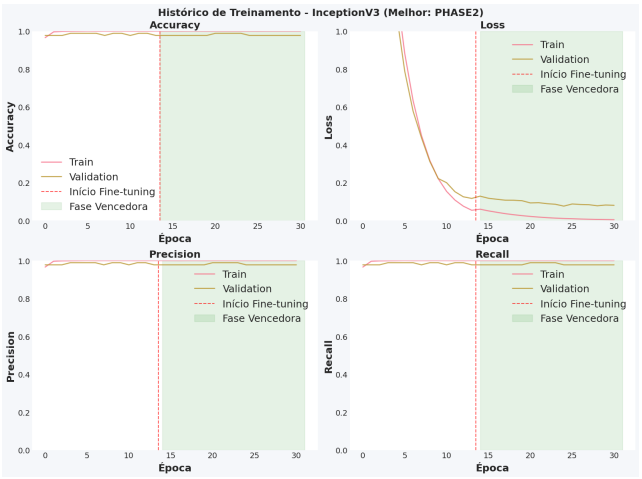


Figura 12. Curvas de treinamento e validação *InceptionV3*

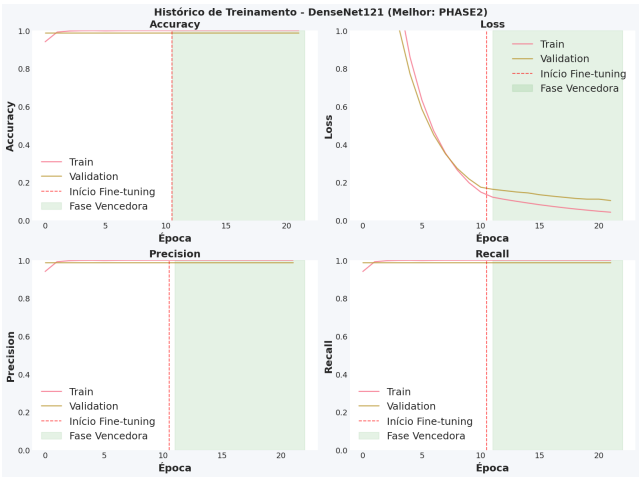


Figura 13. Curvas de treinamento e validação *DenseNet121*

Sobre a convergência e estabilidade, todas as arquiteturas apresentaram convergência rápida e estável.

Em relação ao diagnóstico de overfitting, as arquiteturas apresentaram curvas de treino e validação praticamente sobrepostas, indicando ausência de overfitting e efetividade das estratégias de regularização.

4.3 Comparação dos resultados

A Figura 14 compara o desempenho dos modelos clássicos e CNN nas métricas de avaliação.

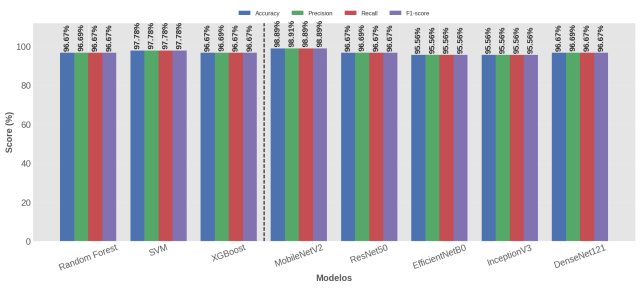


Figura 14. Comparação dos resultados dos modelos

A superioridade das redes neurais convolucionais decorre de diferenças fundamentais na arquitetura e metodologia. Enquanto modelos clássicos requerem extração manual de features, as CNNs aprendem features automaticamente através de camadas convolucionais. Isso permite às CNNs adaptar features exatamente ao problema de classificação de dormentes, capturando características específicas. Além disso, a redução dimensional via PCA aplicada aos modelos clássicos descarta parte da variância, removendo algumas informações que ajudam a dividir as classes, enquanto as CNNs preservam todas as informações relevantes. Além disso, as CNNs beneficiam-se enormemente de transfer learning através do acesso a conhecimento prévio de 1,2 milhões de imagens naturais do ImageNet.

A Figura 15 compara métricas de viabilidade para implementação em plataformas embarcadas entre os modelos clássicos e os modelos de CNN.

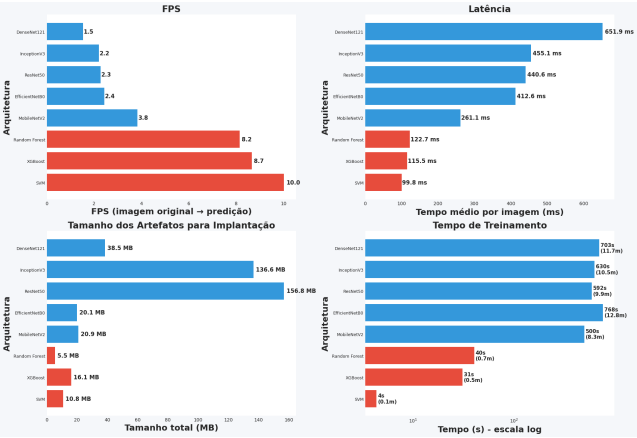


Figura 15. Comparação das métricas de viabilidade embarcada

Os resultados indicam um cenário favorável à utilização dos modelos avaliados em aplicações com processamento local, principalmente considerando os modelos clássicos. Entre eles, o SVM apresentou o melhor desempenho computacional, alcançando 9,4 FPS e a menor latência, além de apresentar menor tamanho. Esses resultados indicam que o modelo possui características interessantes para aplicações que demandem respostas rápidas e menor custo computacional. Entretanto, a viabilidade de sua utilização em um sistema embarcado depende também das características do hardware empregado e de requisitos específicos da aplicação, como memória disponível, capacidade de processamento e tempo de resposta. Dessa forma, os resultados obtidos demonstram um potencial de aplicação embarcada, mas não permitem afirmar que a solução está diretamente pronta para implantação em um dispositivo específico.

5. CONCLUSÃO

Este trabalho modelou, avaliou e comparou os resultados de um sistema automatizado de classificação de qualidade de dormentes ferroviários de concreto mediante arquiteturas clássicas e de deep learning.

A partir dos resultados, foi demonstrado que abordagens baseadas em aprendizado profundo superam os classificadores clássicos dependentes de extração manual de características, embora o desempenho entre eles tenha sido parecido neste tipo de aplicação. A arquitetura MobileNetV2 obteve o melhor desempenho do estudo, registrando acurácia de 98,89%. O mapeamento das métricas computacionais avaliou a viabilidade prática de implementação em sistemas embarcados.

Retornando as 3 questões que nortearam esse trabalho, pode-se concluir:

(1) **RQ1:** Sim, os modelos de aprendizado profundo baseados em redes neurais convolucionais são capazes de classificar a qualidade de dormentes de forma bem robusta e com alta precisão. O pipeline de treinamento utilizado nesse trabalho foi baseado em transfer learning e aplicadas algumas regularizações adicionais. Resultados ainda melhores podem ser atingidos melhorando a qualidade da base de imagens.

(2) **RQ2:** Para este problema em específico, os desempenhos foram similares, sem grandes vantagens em se utilizar os modelos de aprendizado profundo. Porém, em geral, os modelos de aprendizado profundo baseados em redes neurais convolucionais se sobressaem na precisão da classificação. Mas possuem os contrapontos em relação a capacidade computacional. Modelos clássicos bem regularizados podem alcançar resultados satisfatórios com menor custo computacional.

(3) **RQ3:** Entre os modelos clássicos, *SVM* apresenta-se como opção adequada, oferecendo desempenho bom com mínimo tempo de treinamento. Entre os modelos de aprendizado profundo, o modelo *MobileNetV2* demonstra maior viabilidade computacional, sendo uma arquitetura mais compacta e com bom desempenho.

Como limitações deste estudo, destaca-se que o conjunto de dados utilizado, composto por aproximadamente 600 imagens, embora ampliado por meio de técnicas de aumento de dados, ainda restringe a capacidade de generalização do modelo para cenários mais diversos, envolvendo diferentes condições de iluminação, tipos de lastro e variações nos agregados de concreto dos dormentes. Além disso, a abordagem proposta adotou um critério simplificado de classificação, limitando-se à identificação da condição do dormente como ”bom”ou ”ruim”. Dessa forma, trabalhos futuros devem concentrar esforços na ampliação e diversificação da base de dados, incorporando imagens provenientes de diferentes ferrovias e condições operacionais. Adicionalmente, recomenda-se o desenvolvimento de modelos mais robustos, capazes não apenas de identificar a condição geral dos dormentes, mas também de classificar e caracterizar diferentes tipos de defeitos, contribuindo para uma avaliação mais detalhada.

Em suma, os resultados obtidos atestam que a introdução de modelos de machine learning e visão computacional representa um avanço importante na engenharia de manutenção ferroviária. Ao substituir inspeções visuais humanas por um sistema automatizado, abre-se caminho para uma transição definitiva do modelo de manutenção corretiva para uma estratégia preditiva baseada em dados reais, mitigando riscos e otimizando os custos operacionais.

REFERÊNCIAS

Abubakr, M., Rady, M., Badran, K., and Mahfouz, S.Y. (2024). Application of deep learning in damage classification of reinforced concrete bridges. *Ain Shams Engineering Journal*, 15(1), 102297. doi:10.1016/j.asej.2023.102297. URL https://www.sciencedirect.com/science/article/pii/S2090447923001867.

Arafin, P., Billah, A.M., and Issa, A. (2024). Deep learning-based concrete defects classification and detection using semantic segmentation. *Structural Health Monitoring*, 23(4), 2605–2633. doi:10.1177/14759217231168212. URL https://journals.sagepub.com/doi/10.1177/14759217231168212.

Association of American Railroads (2025). Freight rail safety inspections. Website. URL https://www.aar.org/issue/freight-rail-safety-inspections/. Acesso em: 17 fev. 2026.

Associação Brasileira de Normas Técnicas (2014). *NBR 6118: Projeto de Estruturas de Concreto – Procedimento*. ABNT, Rio de Janeiro, RJ. URL https://www.abntcatalogo.com.br/norma.aspx?ID=317027. Norma Técnica Brasileira.

Associação Brasileira de Normas Técnicas (2015). *NBR 11709: Dormentes de Concreto – Projeto, Materiais e Componentes*. ABNT, Rio de Janeiro, RJ. URL https://www.abntcatalogo.com.br/norma.aspx?ID=336502. Norma Técnica Brasileira.

Associação Nacional dos Transportadores Ferroviários (2024). *Setor Ferroviário de Cargas no Brasil: Impactos do PL 4965/2024*. ANTF, Brasília, DF. URL https://www2.camara.leg.br/atividade-legislativa/comissoes/comissoes-permanentes/cvt/apresentacoes-em-eventos/eventos-2025/arquivos-de-eventos-2025/ap-25-10-28-yuri-pontual-antf. Acesso em: 17 fev. 2026.

Breiman, L. (2001). Random forests. *Machine Learning*, 45(1), 5–32.

Chen, T. and Guestrin, C. (2016). Xgboost: A scalable tree boosting system. In *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, 785–794. ACM.

Chen, Y., Liu, W., Cai, Y., and Xu, R. (2022). Deep learning for automatic recognition of concrete crack with image processing. *Advances in Civil Engineering*, 2022, 1–14.

Cortes, C. and Vapnik, V. (1995). Support-vector networks. *Machine Learning*, 20(3), 273–297.

Dalal, N. and Triggs, B. (2005). Histograms of oriented gradients for human detection. *IEEE Conference on Computer Vision and Pattern Recognition*, 886–893.

Deutsche Luft- und Raumfahrtzentrum (2025). Railway sleeper inspection with ai software for deutsche bahn. Website. URL https://event.dlr.de/en/hm2025/bahn-schwelleninspektion/. Acesso em: 17 fev. 2026.

Feng, C., Zhang, H., Wang, S., Li, Y., Wang, H., and Yan, F. (2019). Structural damage detection using deep convolutional neural network and transfer learning. *KSCE Journal of Civil Engineering*, 23(10), 4493–4502. doi:10.1007/s12205-019-0437-z.

Ferdousil, R., Laamarti, F., Yang, C., and Saddik, A.E. (2024). A reusable ai-enabled defect detection system for railway using ensembled cnn. *Applied Intelligence*, 54. doi:10.1007/s10489-024-05676-8. URL https://link.springer.com/article/10.1007/s10489-024-05676-8. Ensemble de VGG19, MobileNetV3 e ResNet50 com 99% de acurácia.

Goodfellow, I., Bengio, Y., and Courville, A. (2016). *Deep Learning*. MIT Press.

Géron, A. (2022). *Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow*. O'Reilly Media, 3 edition.

He, K., Zhang, X., Ren, S., and Sun, J. (2016). Deep residual learning for image recognition. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 770–778. IEEE, Las Vegas, NV, USA. doi:10.1109/CVPR.2016.90.

Hsu, C.W., Chang, C.C., and Lin, C.J. (2003). A practical guide to support vector classification. Department of Computer Science, National Taiwan University.

Huang, G., Liu, Z., van der Maaten, L., and Weinberger, K.Q. (2017). Densely connected convolutional networks. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 4700–4708. IEEE, Honolulu, HI, USA. doi:10.1109/CVPR.2017.243.

International Organization for Standardization (2022). Railway applications – concrete sleepers and bearers for track – part 1: General requirements. International Standard ISO 22480-1:2022, International Organization for Standardization, Geneva, Switzerland.

Jolliffe, I.T. (2002). *Principal Component Analysis*. Springer-Verlag, 2nd edition.

Kaewunruen, S., Adesope, A.A., Huang, J., et al. (2024). Ai-based technology to prognose and diagnose complex crack characteristics of railway concrete sleepers. *Discover Applied Sciences*, 6, 217. doi:10.1007/s42452-024-05880-8. URL https://link.springer.com/article/10.1007/s42452-024-05880-8.

Kornblith, S., Shlens, J., and Le, Q.V. (2019). Do better imagenet models transfer better? In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2661–2671. IEEE, Long Beach, CA, USA. doi:10.1109/CVPR.2019.00277. URL https://arxiv.org/abs/1805.08974.

Kumar, A., Asthana, A., and Parashar, A. (2022). Machine learning based detection and classification of concrete structural damage. *Journal of Infrastructure Systems*, 28(3), 04022020.

Melo, P.M. (2026). Concrete sleeper classification. https://github.com/pedrmelo/concrete-sleeper-classification/tree/main. GitHub repository.

Passos, R.A.d.S.L. (2021). *Aplicação de Redes Neurais Artificiais na Identificação de Defeitos Superficiais em Trilhos Ferroviários*. Dissertação (mestrado em engenharia de transportes), Instituto Militar de Engenharia, Rio de Janeiro, Brasil.

Sandler, M., Howard, A., Zhu, M., Zhmoginov, A., and Chen, L.C. (2018). Mobilenetv2: Inverted residuals and linear bottlenecks. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 4510–4520.

Simonyan, K. and Zisserman, A. (2014). Very deep convolutional networks for large-scale image recognition. *IEEE International Conference on Computer Vision*

(ICCV).

Szegedy, C., Vanhoucke, V., Ioffe, S., Shlens, J., and Wojna, Z. (2016). Rethinking the inception architecture for computer vision. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2818–2826.

Tan, M. and Le, Q.V. (2019). Efficientnet: Rethinking model scaling for convolutional neural networks. In *Proceedings of the 36th International Conference on Machine Learning (ICML)*, 6105–6114.

Tang, Y. and Qian, Y. (2024). High-speed railway track components inspection framework based on yolov8 with high-performance model deployment. *High-speed Railway*, 2(1), 42–50. doi:10.1016/j.hspr.2024.02.001. URL `https://www.sciopen.com/article/10.1016/j.hspr.2024.02.001`.

Utami, W. (2023). Concrete sleepers dataset. URL `https://www.kaggle.com/datasets/wachyuutami/concrete-sleepers/data`. Acesso em: 06 abr. 2026.

Wang, T., Yang, F., and Tsui, K.L. (2020). Real-time detection of railway track component via one-stage deep learning networks. *Sensors*, 20(15), 4325. doi:10.3390/s20154325. URL `https://www.mdpi.com/1424-8220/20/15/4325`.